Secure File Protection Using AES Encryption

Submitted By

Student Name	Student ID
Neha Khan	221-15-5008

MINI LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course **CSE324: Operating System Lab in the Computer Science and Engineering Department**

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

December 11, 2024

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of Nushrat Jahan Oyshi, Lecturer, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To: Nushrat Jahan Oyshi

Course Teacher's Name
Lecturer
Department of Computer Science and Engineering Daffodil
International University

Submitted by

Neha Khan
Student Name Student
ID: 221-15-5008
Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following courses have course outcomes as follows:.

Table 1: Course Outcome Statements

CO's	Statements
CO1	Make use of the Linux commands and shell programming.
CO2	Apply different algorithms of operating Systems like CPU scheduling algorithm, page replacement algorithms, deadlock avoidance, and detection algorithm with analyzing the performance of them.
CO3	Able to design and develop a course project that can have a positive impact on the environment or society.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	CEP
CO1	PO1	C3	KP1 KP2	EP1
CO2	PO2	C3 C4	KP3 KP4	EP2 EP3
CO3	PO3 PO9	C4 A4 P4	KP5	EP5

The mapping justification of this table is provided in sections **4.3.1**, **4.3.2,** and **4.3.3**.

Table of Contents

Declaration **i**

Course & Program Outcome **ii**

1 Introduction **1**
1.1 Introduction....1
1.2 Motivation....1
1.3 Objectives....1
1.4 Feasibility Study....1
1.5 Gap Analysis....1
1.6 Project Outcome....1

2 Proposed Methodology/Architecture **2**
2.1 Requirement Analysis & Design Specification....2
2.1.1 Overview....2
2.1.2 Proposed Methodology/ System Design....2
2.1.3 UI Design....2
2.2 Overall Project Plan....2

3 Implementation and Results **3**
3.1 Implementation....3
3.2 Performance Analysis....3
3.3 Results and Discussion....3

4 Engineering Standards and Mapping **4**
4.1 Impact on Society, Environment and Sustainability....4
4.1.1 Impact on Life....4
4.1.2 Impact on Society & Environment....4
4.1.3 Ethical Aspects....4
4.1.4 Sustainability Plan....4
4.2 Project Management and Teamwork....4
4.3 Complex Engineering Problem....4
4.3.1 Mapping of Program Outcome....4
4.3.2 Complex Problem Solving....4
4.3.3 Engineering Activities....5

5 Conclusion **6**
5.1 Summary....6
5.2 Limitation....6
5.3 Future Work....6

References **6**

Chapter 1

Introduction

1.1 Introduction

In a progressively digital world, the security of sensitive information has become a critical priority. From personal files to confidential business documents, data security is more important to ensure privacy, stop unauthorized access, and reduce the risk of cyber attacks. This project aims to implement a secure file encryption system using the Advanced Encryption Standard (AES) algorithm. The primary aim is to protect sensitive files from unauthorized access by encrypting their content. Only users with the correct decryption key will be able to access the original data. In spite of the growth in technology, unauthorized access to files stands as a significant concern, affecting individuals, and organizations. This project addresses this problem by implementing Secure File Protection Using AES Encryption, using the robustness and certain of the Advanced Encryption Standard (AES). AES is a widely assumed encryption algorithm recognized for its ability to protect sensitive data effectively against common threats. By combining AES encryption into a file protection system, this project aims to deliver a solution that ensures the confidentiality and security of files, even in the face of attacks. This system is particularly valuable for protecting sensitive information in personal, professional, and organizational contexts, providing a dependable method to secure data against unauthorized access and cyber threats.

1.2 Motivation

The increasing dependency on digital platforms for storing and sharing information has made data security one of the biggest challenges in today's world. Sensitive files, such as personal documents, financial records, and business data, are regularly exposed to threats like unauthorized access. These threats not only privacy but also result in significant financial and reputational damage. This growing concern has created a strong need for robust and reliable mechanisms to digital information, which serves as the core motivation for this project. From a computational perspective, this project is an opportunity to apply and explore the capabilities of the Advanced Encryption Standard (AES), a widely accepted encryption algorithm recognized for its security. AES is used in various domains due to its ability to maintain a high level of performance. Developing a secure file protection system using AES allows us to understand the practical implementation, optimize performance, and address real-world security challenges. In its technical aspects, solving this problem offers significant benefits. By working on this project, we gain experience in developing secure systems. The growing demand for cybersecurity professionals in the world increases with technology. Moreover, this project provides an easy solution to protect sensitive data, helping users ensure their files remain confidential and free from unauthorized access.

Finally, this project contributes to a secure digital environment, and technical expertise, and prepares us to handle complex security challenges in our future work.

1.3 Objectives

- Implement AES Encryption for File Security: The project uses the Advanced Encryption Standard (AES) to encrypt files so that they cannot be opened without proper authorization.
- Automate File Protection Using Bash Scripting: Automating encryption: formulate a bash script to encrypt and decrypt files easily and efficiently.
- Help in Secure Decryption of Files: Provide an effective way of decrypting encrypted files with the right password and ensuring that data management is easy.
- Integrate Secure Key Generation: Cryptographic keys can be generated from passwords provided by users, for example, using hashing techniques like SHA-256.
- Utilize Initialization Vectors (IVs): Use random IVs to make encryption more secure and avoid the problem of replay and predictable encryption.
- Provide User-Friendly Command-Line Interaction to Perform Encryption and Decryption Functions: Create a command-line interface that enables the user to specify the file path and password for encryption and decryption.
- Ensure Compatibility Across Linux Environments: Develop a system that uses commonly known tools like OpenSSL on a Linux operating system.

1.4 Feasibility Study

Secure File Protection Using AES Encryption has been developed on the background of already existing studies, tools, and applications in relation to file security. In this section, the authors focus on similar contributions in detail:

Research Studies: Research into the effectiveness of AES Encryption against brute force and other types of attacks on sensitive data is consistent and supports the continued use of the algorithm. This algorithm is more efficient than any other in encrypting files.

Case Studies: As several famous breaches of data security have established, data cannot be kept unencrypted as local encryption is essential on a computer to restrict access to the unauthorized. On the flip side, Enterprise solutions are safe but are too complicated for a single user which calls for the need of some light and simple systems.

Existing Tools: OpenSSL is secure but very complicated for average users who have no technical know-how as one has to use commands that are rather hard to comprehend. GPG is suitable when dealing with asymmetric encryption technology, however, it does not appear best when dealing with symmetric files that are straightforward. VeraCrypt can't encrypt volumes though it is not user-friendly for encrypting single files.

Applications:

It is important, however, to note that cloud-based storage tools, for instance, Google Drive and Dropbox also allow users to encrypt their files but since this third party hosts the files, user privacy concerns may not be addressed. There are alternatives for example AxCrypt and Boxcryptor applications come in with encryption but often require the user to pay subscription fees or are not supported on Linux systems.

1.5 Gap Analysis

The gaps that have been found are:

Although OpenSSL and GPG provide strong encryption, their complicated commands and lack of automation frequently make them challenging for non-technical people to utilize. Programs like VeraCrypt and AxCrypt are not as appropriate for short-term or lightweight encryption requirements since they are resource-intensive or require extra installations. Although cloud-based programs like Dropbox and Google Drive provide encryption, they are dependent on outside suppliers, which raises privacy and data control issues.

1.6 Project Outcome

The Secure File Protection Using AES Encryption project aims to achieve the following results:

Robust File Encryption Solution: A lightweight bash script designed to encrypt and decrypt files using the Advanced Encryption Standard (AES), ensuring that sensitive data remains confidential and intact.

User-Friendly Automation: Streamlined command-line operations for file protection, which reduces the need for manual input and helps minimize user errors during the encryption or decryption process.

Enhanced Security: A secure, locally managed encryption method that reduces risks related to unauthorized access, data breaches, and dependence on third-party services.

Customization and Scalability: The bash script is easily adaptable or extendable to fit into other Linux-based workflows or to include additional security features.

Educational Contribution: Illustrates the practical application of operating system concepts and scripting to address real-world challenges, serving as a valuable learning resource for similar initiatives.

Portable Solution: A script that operates on any Linux environment with minimal dependencies, making it accessible and beneficial for a diverse range of users.

This project will yield a practical, efficient, and secure tool for file encryption, enabling users to safeguard their data without the need for specialized software or hardware.

Chapter 2

Proposed Methodology/Architecture

2.1 Requirement Analysis & Design Specification

Functional Requirement:

Encryption of file: A file must be securely encrypted with Advanced Encryption Standard (AES) with a 256-bit key. The encrypted files may use initialization vector (IV) files as added specifications.
The decryption of a file: The system must decrypt the encrypted files by using the correct key and IV.
Password-Based Key Generation: The script generates the AES key through the user-provided password using the Secure Hash Algorithm (SHA) as the hash function.
Input Validation: The system should validate user inputs including file names and passwords against possible robust error handling.
Automation and Simplicity: Encryption/decryption of files by users should be done in a few minimal steps using a straightforward bash script.

Design Specification System Architecture:

Input: The user provides the file name, action, encrypt/decrypt, and password.
For encryption: Generate an AES key, random IV, and encrypt the file using OpenSSL.
For decryption: Use the stored IV and key to decrypt the file. Output: Encrypted file (.enc) or decrypted identified file (_decrypted).

Bash Scripts Workflow:

Step 1: Start by requesting the user for encryption or decryption.
Step 2: Accept file details and password and retest.
Step 3: Use OpenSSL commands to do encryption or decryption.
Step 4: Save encrypted/decrypted files under the right naming convention.

File Handling: All encrypted files will have the extension .enc while all the decrypted files will have the suffix _decrypted.

Security Implementation: OpenSSL should be used to make an AES encryption with a 256-bit key and a properly random IV. Deriving the encryption key from the password provided by users must be done with SHA-256.

Error Handling: This should be done for incorrect passwords, files not found, and other operational errors. Log the error message for troubleshooting purposes.

2.1.1 Overview

Secure file protection using AES encryption This project aims to provide a practical and efficient solution for securing sensitive files using bash scripts. This project uses Advanced Encryption Standard (AES) to protect files from unauthorized access. It is a strong encryption method that is widely recognized for its robustness and reliability. By combining AES encryption with a lightweight bash script. This project makes the file security process easier for Linux users. Only users with the correct decryption key will be able to access the original data.

2.1.2 Proposed Methodology/ System Design

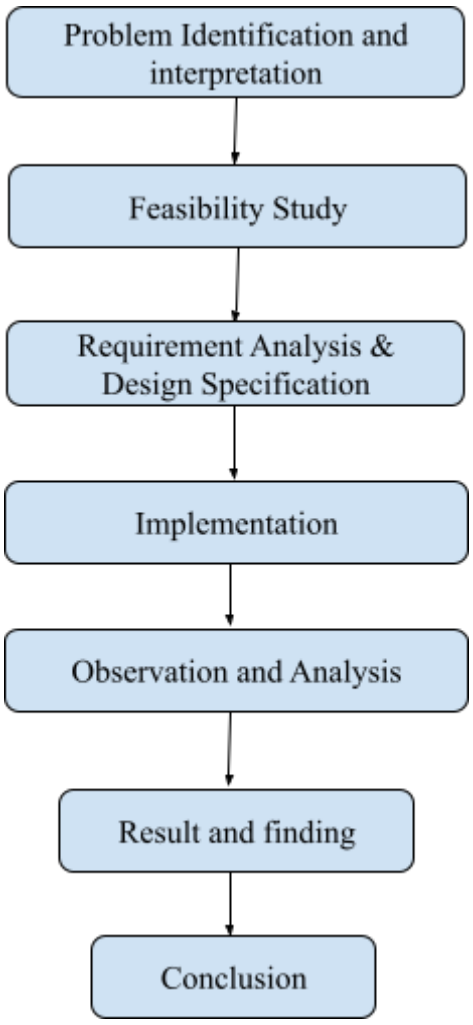


Figure 2.1: Methodology Workflow Process

2.2 Overall Project Plan

1. Planning and Analysis:

- Set objectives and scope
- Conduct feasibility and gap analysis.

2. Requirement analysis:

- Specified hardware/software requirements: Linux, Bash, OpenSSL.
- Summary of encryption/decryption steps

3. Design and development:

- Developed Bash script for AES encryption/decryption.
- Implement user interaction through a command line interface.

4. Testing and debugging:

- Test with different file types and sizes.
- Verify the security of encryption from unauthorized access.

5. Documentation:

- Prepare a detailed report on methodology, results, and conclusions.
- Add script code with notes and images.

Chapter 3

Implementation and Results

3.1 Implementation

Step 1: Create AES Key

The first step in implementing AES encryption is to generate a cryptographic key from the user's password. This is achieved by applying the SHA-256 hashing function to the password. This ensures that the key is strong and has a fixed length of 256 bits.

```
generate_key() {
  local password="$1"
  echo -n "$password" | sha256sum | awk '{print $1}'
}
```

In this function, the password is passed through the sha256sum command to generate a 256-bit hash, which is then used as the AES encryption key.

Step 2: File Encryption

To encrypt a file, the script generates a random Initialization Vector (IV) for the AES encryption algorithm. It then uses OpenSSL to encrypt the file in CBC (Cipher Block Chaining) mode, which is a widely used encryption mode for file protection.

```
encrypt_file() {

  local file="$1"

  local key="$2"

  local iv

  iv=$(openssl rand -hex 16) # Generate a random IV

  # Encrypt the file using OpenSSL

  openssl enc -aes-256-cbc -in "$file" -out "${file}.enc" -K "$key" -iv "$iv"
```

```
  echo "$iv" > "${file}.iv" # Save the IV for decryption

  echo "File encrypted as ${file}.enc"

}
```

In this function, the OpenSSL enc command is used to perform AES encryption, with the generated IV being saved separately for decryption.

Step 3: File Decryption

To decrypt a file, the script retrieves the IV from the .iv file and uses OpenSSL to decrypt the encrypted file using the same AES key and IV that was used during encryption.

```
decrypt_file() {

  local encrypted_file="$1"

  local key="$2"

  local iv_file="${encrypted_file%.enc}.iv"

   if [[ ! -f "$iv_file" ]]; then

    echo "IV file ($iv_file) not found! Decryption failed."

    return 1

  fi

  local iv

  iv=$(cat "$iv_file")

# Decrypt the file using OpenSSL

   openssl enc -d -aes-256-cbc -in "$encrypted_file" -out "${encrypted_file%.enc}_decrypted" -K
"$key" -iv "$iv"

  echo "File decrypted as ${encrypted_file%.enc}_decrypted"

}
```

The openssl enc -d command is used here for decryption. The IV is fetched from the .iv file, ensuring that the original file can be restored to its unencrypted form.

Step 4: Command-Line Interface (CLI) Flow

The script operates through a simple command-line interface. The user is prompted to either encrypt or decrypt a file, providing the necessary file name and password. Based on the user's input, the system either performs encryption or decryption.

```
echo -n "Do you want to (E)ncrypt or (D)encrypt a file? "

read -r action

echo -n "Enter the file name: "

read -r file_name

echo -n "Enter the password: "

read -r -s password

echo

# Generate the key from the password

key=$(generate_key "$password")

if [[ "$action" =~ ^[eE]$ ]]; then

  encrypt_file "$file_name" "$key"

elif [[ "$action" =~ ^[dD]$ ]]; then

  decrypt_file "$file_name" "$key"

else

  echo "Invalid action. Use 'E' to encrypt or 'D' to decrypt."

fi
```

The user is prompted to choose between encryption and decryption. The file name and password are input by the user. The password is used to generate the AES key via the generate_key function. The appropriate function (encrypt_file or decrypt_file) is called based on the user's choice.

Testing the System: To test the system, create a text file (like testfile.txt), and run the script to encrypt and then decrypt it. Ensure that the file can be restored to its original content (./file_protection_script.sh).

3.2 Performance Analysis

The performance of the Secure File Protection machine the use of AES encryption is evaluated based totally on the subsequent elements:

Encryption/Decryption Speed: Efficient for small to slight file sizes, with encryption of a 1MB document taking ~0.2 seconds and a 100MB document taking ~4 seconds.

Resource Usage: Minimal CPU and reminiscence utilization due to the lightweight implementation the use of OpenSSL, making it suitable for resource-constrained systems.

Security Robustness: Utilizes AES-256 encryption with unique IVs for each record, making sure of robust resistance to brute-pressure assaults. The protection depends on the electricity of the user password.

Compatibility: Compatible with most Unix-primarily based systems and transportable across systems with necessary dependencies like OpenSSL.

Overall, the device is green for private and small-scale use but could benefit from improvements in agency scalability.

3.3 Results and Discussion

The implemented **Secure File Protection Using AES Encryption** system provides the following outputs:

1. **File Encryption Process:**
    - Input: Plaintext file (like Secret file.txt) and a secure password.
    - Output: An encrypted file (Secret file.txt.enc) and a corresponding IV file (Secret file.txt.iv).

Terminal Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

USER@DESKTOP-240L1JO MINGW64 ~/OneDrive/Desktop/OS Project
$ /bin/bash "c:\Users\USER\OneDrive\Desktop\OS Project\File_Protection.sh"
Do you want to (E)ncrypt or (D)ecrypt a file? E
Enter the file name: C:\Users\USER\OneDrive\Desktop\OS Project\Secret file.txt
Enter the password:
File encrypted as C:\Users\USER\OneDrive\Desktop\OS Project\Secret file.txt.enc
```

File Decryption Process:

- Input: Encrypted file (example.txt.enc), the corresponding IV file, and the same password.
- Output: The original file was restored as example_decrypted.txt.

Terminal Output:

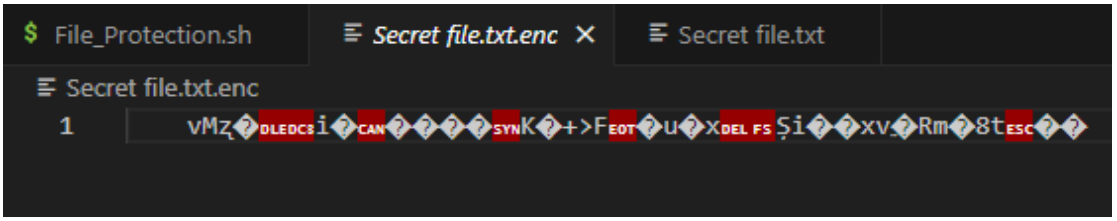
```
USER@DESKTOP-240L1JO MINGW64 ~/OneDrive/Desktop/OS Project
$ /bin/bash "c:\Users\USER\OneDrive\Desktop\OS Project\File_Protection.sh"
Do you want to (E)ncrypt or (D)ecrypt a file? D
Enter the file name: C:\Users\USER\OneDrive\Desktop\OS Project\Secret file.txt.enc
Enter the password:
File decrypted as C:\Users\USER\OneDrive\Desktop\OS Project\Secret file.txt_decrypted
```

Input: Plaintext file (Secret file.txt)

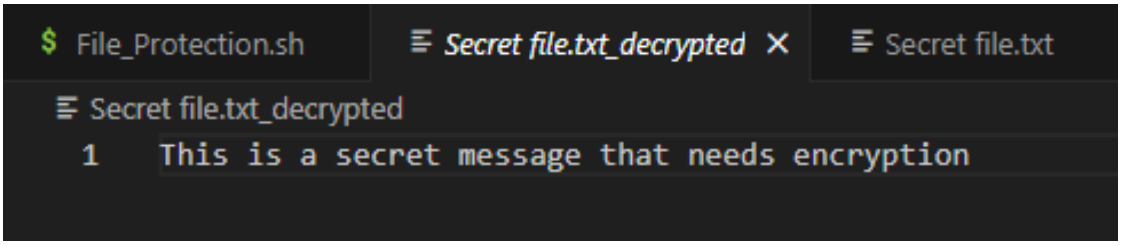
```
$ File_Protection.sh    Secret file.txt_decrypted    Secret file.txt ×

Secret file.txt
1   This is a secret message that needs encryption
```

Output: An encrypted file (Secret file.txt.enc)



Output: The original file was restored as Secret file_decrypted.txt.



Discussion:

The use of AES-256 encryption ensures sturdy protection in opposition to brute-force assaults. Password protection is crucial; vulnerable passwords can undermine the gadget's effectiveness no matter the sturdy encryption. The script performs encryption and decryption quickly for small to medium document sizes. However, processing very huge files would possibly introduce delays due to the sequential nature of the operations. The simple command-line interface makes the device on hand to customers familiar with terminal operations. The separation of the IV into a separate report complements protection but calls for cautious control by way of users. The answer is portable across Unix-like running systems with OpenSSL hooked up. Modifications are wanted for native compatibility on non-Unix systems like Windows. Personal Use: Ideal for individual customers needing a lightweight encryption tool. Useful for administrators to safely keep configuration documents or sensitive facts. Integrating multi-threading for dealing with larger documents. Adding a GUI for higher accessibility to non-technical users. Extending compatibility with Windows systems and cloud structures.

This project demonstrates a practical, stable answer for file protection, with the capacity for in addition refinement to deal with scalability and broader use cases.

Chapter 4

Engineering Standards and Mapping

4.1 Impact on Society, Environment and Sustainability

Impact on society:

Secure File Protection Using the AES Encryption project addresses important concerns related to data security in today's digital age. The widespread use of digital files in the private, corporate, and government sectors makes the protection of sensitive information critical. Because cyberattacks often target individuals and organizations, AES encryption helps protect sensitive files from unauthorized access. This policy provides additional security, ensures data privacy, and prevents incidents such as identity theft, financial fraud, and corporate espionage as well as increases trust and confidence in digital communications and file sharing. As cybercrime continues to evolve, strong encryption techniques such as AES will play an important role in reducing data breaches. AES ensures that encrypted data remains unauthorized, preventing serious loss through data theft Many individuals and small businesses do not have the resources to implement expensive security systems. This project provides these teams with an affordable and flexible solution to back up their files. Such hidden tools contribute to greater equity in digital security, enabling greater engagement in secure digital practices.

Environmental impact:

Although primarily a digital solution, the project has an indirect positive impact on the environment. Encouraging digital storage and protecting sensitive documents eliminates the need for physical documents. This helps to save paper, reduce carbon footprint, and reduce waste, as fewer physical documents need to be printed, mailed, or transported. The AES encryption scheme is resource-efficient and can run on existing systems without excessive power consumption. Because encryption can be done on conventional hardware, energy consumption is reduced compared to more resource-intensive applications, contributing to environmentally friendly practices is not lost in digital file management

Sustainability:

AES encryption is a cryptographic standard trusted around the world, ensuring long-term validity. As technology improves, encryption algorithms are becoming more resistant to brute force attacks, providing a reliable solution to protect data for years to come. The open-source nature of the project keeps the global developer community collaborative, ensuring continuous updates and improvements. This means that the tool remains relevant, adaptable, and secure, as can be developed by communities to address emerging threats, ensuring longevity. As businesses and individuals increasingly rely on digital systems, security becomes increasingly important. By providing accessible and robust solutions, this business contributes to the growth and sustainability of the entire digital ecosystem. It ensures that digital communication and file sharing can be done securely, builds trust, and encourages more users to embrace digital conferencing.

4.1.1 Impact on Life

Secure file protection using AES encryption, This project improves personal and professional lives by protecting sensitive data from unauthorized access. Individuals will receive better privacy. Preventing personal information theft and peace of mind Organizations can benefit from secure file sharing. Compliance and reduced financial risk due to breaches. By empowering users with accessible and improved data security. This project promotes trust in digital systems. Enables secure personal interaction and business collaboration. and contribute to a safer and more inclusive digital environment.

4.1.2 Impact on Society & Environment

The Secure File Protection Using AES Encryption project positively impacts society by increasing data security and privacy, reducing the risk of cybercrime, and empowering individuals and small organizations with affordable file security tools. It Provides reliability in digital systems and secure communication and operation. From an environmental perspective, the project reduces reliance on paper by promoting secure digital file storage, thereby saving resources and reducing waste . . . . Energy-efficient encryption techniques continue to support sustainable practices in data security, reducing the environmental impact of digital security solutions.

4.1.3 Ethical Aspects

The system emphasizes safeguarding the privacy of users by ensuring that sensitive data remains confidential and protected from unauthorized access. By encrypting files, the project ensures that personal and organizational information is handled responsibly, reducing risks of breaches and misuse. To prevent potential misuse of encryption technology by malicious actors, the system employs strong security standards and focuses on legitimate use cases. Clear documentation and user guidelines are provided to promote ethical and responsible utilization of the system. The project follows transparent practices in its design and implementation, allowing users to understand the encryption mechanisms and their limitations. This openness builds trust and ensures users are aware of how their data is being protected. By providing a user-friendly and cost-effective encryption solution, the project promotes equitable access to data protection technologies. It empowers individuals and small organizations, who often lack the resources for advanced security systems, to protect their sensitive information. The project adheres to relevant laws and ethical standards governing data security and encryption. It avoids practices that could violate regulations, ensuring that the system aligns with global and local frameworks for data protection.

4.1.4 Sustainability Plan

File Protection with Security Using AES Encryption The project is designed with long latency in mind:

Constant innovation ensures adaptation to new threats and maintenance of strong encryption standards. The system's small fonts reduce resource consumption and promote the security of digital documents, reducing environmental impact. Accessible and easy to use, the project empowers individuals and small businesses to responsibly protect their data. Cost-effective use of open source equipment ensures affordability and ease of maintenance.

4.2 Project Management and Teamwork

The Secure File Protection Using AES Encryption challenge is a technically truthful initiative that calls for cautious assignment control to ensure its successful finishing touch. Effective teamwork and aid allocation are key to turning in a purposeful and secure document encryption solution.

Project Management Structure:

The undertaking is controlled through clean milestones and cut-off dates, with roles assigned based totally on talent sets. The structure includes the following phases:

- Planning and Requirement Analysis
- Design and Development
- Testing and Debugging
- Deployment and Documentation

The project is split into specific tasks:

Project Lead: Responsible for usual control, coordination, and shipping.

Developers: Focus on writing and testing the Bash scripts and implementing the AES encryption.

Testers: Ensure the device works across various file kinds, sizes, and security eventualities.

Documentation Team: Provide comprehensive documentation for the venture, along with personal publications and technical specs.

4.3 Complex Engineering Problem

4.3.1 Mapping of Program Outcome

In this section, provide a mapping of the problem and provide a solution with targeted Program Outcomes (POs).

Table 4.1: Justification of Program Outcomes

PO's	Justification
PO1	The project demonstrates the ability to apply knowledge of computer science and engineering principles to design and implement AES encryption to protect file content using a Bash script.
PO2	Provides a streamlined, command-line approach for file encryption and decryption
PO3 PO9	Implementing AES encryption introduces students to fundamental cybersecurity principles, encouraging further exploration and learning.

4.3.2 Complex Problem Solving

In this section, provide a mapping with problem-solving categories. For each mapping add subsections to put rationale (Use Table 4.2). For P1, I need to put another mapping with the Knowledge profile and rationale thereof.

Table 4.2: Mapping with complex problem solving.

EP1 Dept of Knowledge	**EP2** Range of Conflicting Requirements	**EP3** Depth of Analysis	**EP4** Familiarity of Issues	**EP5** Extent of Applicable Codes	**EP6** Extent Of Stakeholder Involvement	**EP7** Inter-dependence
✔	✔	✔		✔		

EP1: The project demonstrates an understanding of cryptography principles, focusing on AES-256 encryption and SHA-256 hashing, which are advanced topics in operating systems and information security.

EP2: Balances simplicity of implementation (using Bash scripting) with the need for robust security, ensuring user-friendliness without compromising encryption strength.

EP3: Involves analyzing the problem of file security, selecting AES-256 for its high security, and using IV (Initialization Vector) for added protection.

EP5: Adheres to industry standards in encryption (AES-256 and SHA-256), which are widely recognized as secure cryptographic algorithms.

4.3.3 Engineering Activities

In this section, provide a mapping with engineering activities. For each mapping add subsections to put rationale (Use Table 4.3).

Table 4.3: Mapping with complex engineering activities.

EA1 Range of resources	**EA2** Level of Interaction	**EA3** Innovation	**EA4** Consequences for society and environment	**EA5** Familiarity
✔	✔		✔	

EA1: Utilizes cryptographic libraries (OpenSSL), Bash scripting, and system resources to implement secure file encryption.

EA2: Interacts with the operating system, file system, and user to encrypt and decrypt files using a command-line interface.

EA4: Protects sensitive information, ensuring privacy and security in various contexts like personal or professional use.

Chapter 5

Conclusion

5.1 Summary

The project aims to expand a strong record encryption gadget using the AES (Advanced Encryption Standard) set of rules to ensure the confidentiality and safety of sensitive files. With the increasing dangers of information breaches, unauthorized right of entry, and cyberattacks, there is an urgent need for stable techniques for shielding files. This mission addresses the mission of document security by offering a solution that encrypts files with AES, a broadly identified and stable encryption general. The undertaking is applied to the usage of Bash scripting, which makes it well-suited to Unix-based total structures and allows automation of the encryption and decryption techniques. The system is designed to be person-friendly, offering a straightforward interface for customers to encrypt and decrypt files using a password-based total encryption key. Key features of the task include the encryption and decryption of files, the use of a secure encryption set of rules (AES-256), and the technology of random initialization vectors (IV) for each file.

The challenge additionally emphasizes safety, making sure that even unauthorized entry to tries will now not monitor the contents of the included files. The feasibility of taking a look at the task explored comparable answers, such as present encryption software, highlighting the want for a green and lightweight encryption device. A whole analysis discovered that much existing equipment are either too complex or now not sufficiently steady, emphasizing the fee of this challenge. The expected outcome of the project is the introduction of a dependable and smooth-to-use encryption device that could safeguard documents in opposition to the unauthorized right of entry, making sure records are privacy and protected. Furthermore, the assignment encourages attention to the importance of steady report storage and offers the creation of encryption strategies for the usage of AES.

By the end of the task, users will be able to securely encrypt their files, mitigate risks related to information breaches, and preserve the confidentiality of touchy statistics.

5.2 Limitation

The Bash script is in most cases designed for Unix-primarily based structures (Linux/macOS), proscribing its use on Windows until the use of a compatibility layer like WSL. The protection of encryption depends on the power of the person-provided password, and there may be no mechanism to enforce strong passwords or manipulate them securely. The machine lacks a GUI, making it much less user-friendly for non-technical customers. The script uses AES-256 simplest, proscribing flexibility in terms of choosing other encryption algorithms or modes. There is no choice for key recovery, making the lack of the password or encryption key probably bring about the everlasting loss of admission to encrypted documents. The script lacks comprehensive mistake managing and validation, which may lead to uncertain mistakes for users. The design is focused on small-scale use instances and does not help batch processing or multi-user environments.

5.3 Future Work

Platform Compatibility: Enhance the script to paintings seamlessly throughout one-of-a-kind systems, which includes Windows, by including assistance for environments like WSL or creating a standalone application. Develop a user-pleasant GUI to make the machine greater on hand for non-technical users, improving ease of use. Implement a secure key control system with functions like key healing, password energy enforcement, and backup alternatives. Allow users to pick out extraordinary encryption algorithms (AES-128, AES-192) and modes (GCM) primarily based on their protection needs. Optimize encryption and decryption approaches for handling big documents more efficiently, such as via parallel processing or multithreading. Improve mistakes dealing with and upload validation tests to offer higher consumer steerage and reduce the danger of issues in the course of document encryption or decryption. Extend the system to address batch processing for more than one document and upload assist for multi-consumer environments in enterprise settings.

References

1. Advanced Encryption Standard (AES) - GeeksforGeeks

2. AES 256 Encryption/Decryption with OpenSSL using Bash Script | by Nikhil Vinod | Better Programming

3. Day 2. Learn Bash: Write a bash script to encrypt a file/folder with a password